Weekly Progress Report

Project: Custom 2D Platformer - Group 3
Date / Week: 6/7 to 11/7 | Week 05

1. Weekly Overview

- **Primary Goal:**
  - Implement core physics (gravity, basic collision) and integrate complex collision detection (e.g., Mario stomping on Goombas).
  - Finalize Mario's movement controls (keyboard inputs) and size-changing logic using the Strategy Pattern.
  - Develop the tilemap renderer to display static levels (bricks, pipes) from text files.
  - Spawn classic items (Mushroom, Coin) and enemies (Goombas, Koopas), and implement their basic behaviors (walking/bouncing).
- **Completion Rate:** 10% of the weekly plan completed.
- **Note:** This week, members reviewed and prepared for the tests. Workflow is decided to be delayed one week.

2. Individual Task Breakdown

25125057 - Trần Minh Khoa is responsible for the **Core Engine & Physics**.

- **Completed Tasks:** no completed tasks this week

25125045 - Trần Như Khải is in charge of the **Hero & Items Logic**.

- **Completed Tasks:** *Implemented the core movement and jumping mechanics for the Mario character, accurately mapping keyboard inputs to coordinate velocity and managing the grounded state logic.

  *Created a custom Animator class to handle sprite rendering and frame updates. This system is designed based on a Real-time State Machine model dedicated specifically to animations, allowing seamless and dynamic transitions between various character states (e.g., Idle, Walking, Jumping) during the game loop.

- **Proof:** https://github.com/trannhukhai98438/CS202-Group03-2DGame/tree/feature/Hero

25125024 - Đỗ Viết Hoàng Long is handling the **Enemies & AI Design**.

- **Completed Tasks:** no completed tasks this week

25125056 - Trần Đăng Khoa is assigned to the development of the **Tilemap, HUD & Audio**.

- **Completed Tasks:** no completed tasks this week

3. Challenges & Solutions

Issue 1: Manually storing and switching individual sprites for various character states (e.g., move, jump, dead) significantly complicated the coding logic. Furthermore, having each object store multiple individual sprite assets led to performance inefficiencies, causing stuttering and unsmooth transitions between animation frames. **Solution 1:** We adopted a sprite sheet approach, combining all character animations into a single large image texture. To manage this efficiently, we implemented a custom Animator class that parses the sprite sheet, stores the specific frame coordinates, and dynamically renders the correct animation sequence based on the character's current active state.

4. AI Usage Notes

- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 15:00 on July 11, 2026, prompt: "Currently, *Character* have attributes like sprite and texture. However, for Heroes and Enemies, they have multiple states (standing, walking, jumping, dead), requiring multiple sprites for each state. Is declaring sprites in the base *Character* class the optimal design approach?" to review approach and find optimal solution, AI suggested Sprite Sheet approach, Student reviewed it and accepted new approach.

- Gemini. Gemini 3.1 Pro, Google, gemini.google.com, accessed 16:00 on July 11, 2026, prompt: "Help me to implement Animator class for parsing sprite sheet, storing frames and rendering correct animation based on Character's state", AI implemented, Student reviewed codes and accepted it.

5. Next Week's Action Plan

- **Core Mechanics:**
    - Implement basic gravity/collision and integrate complex collision detection (e.g., Mario stomping on Goombas).
    - Finish Mario's size-changing logic.
- **Gameplay / Graphics:**
    - Develop the tilemap renderer to display a static Mario level (bricks, pipes) from a text file.
    - Spawn classic items (Mushroom, Coin).
    - Spawn Goombas/Koopas and implement basic walking/bouncing behavior.
- **Expected Deadline:** 18/7/2026